

Heverton Eduardo Peres

# **As 31 Ferramentas Built-in: O Arsenal Completo & Edições Hashline: Precisão com Menos Tokens**

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil  
2026

## Sumário

---

|                                                      |    |
|------------------------------------------------------|----|
| As 31 Ferramentas Built-in: O Arsenal Completo ..... | 3  |
| Abrindo o arsenal do estaleiro .....                 | 3  |
| Três categorias de ferramentas .....                 | 3  |
| Ferramentas de Arquivo (7 ferramentas) .....         | 3  |
| Ferramentas de Runtime (2 ferramentas) .....         | 4  |
| Ferramentas Avançadas (5+ ferramentas) .....         | 5  |
| As 31 ferramentas em detalhe .....                   | 6  |
| Os 16 esquemas internos de URI .....                 | 8  |
| Fluxo de uso combinado .....                         | 8  |
| Economia de tokens na prática .....                  | 9  |
| Próximos Passos .....                                | 10 |
| Edições Hashline: Precisão com Menos Tokens .....    | 11 |
| O problema que consome recursos .....                | 11 |
| O fluxo tradicional: tokens desperdiçados .....      | 11 |
| A solução: hashes como âncoras .....                 | 12 |
| Como funciona o hash .....                           | 12 |
| Comparação com str_replace .....                     | 12 |
| A economia por modelo .....                          | 13 |
| Estrutura de um hash .....                           | 13 |
| Sintaxe de edição .....                              | 13 |
| Exemplo prático .....                                | 14 |
| Implementação no Oh My Pi .....                      | 15 |
| Quando usar hashline edits .....                     | 15 |
| Caso de uso: refatoração de código .....             | 16 |
| Dicas do Mestre de Estaleiro .....                   | 16 |
| Próximos Passos .....                                | 17 |

## As 31 Ferramentas Built-in: O Arsenal Completo

---

### Abrindo o arsenal do estaleiro

---

No capítulo anterior, você dominou o TUI — a interface de comando que traduz cada ação do agente em cards navegáveis. Agora é hora de abrir o arsenal completo: as 31 ferramentas built-in que o agente usa para ler, escrever, buscar, executar, depurar e até controlar seu computador.

Assim como um estaleiro de navios não se constrói com apenas um martelo e uma chave inglesa, um coding agent não se limita a ler e escrever arquivos.

O OMP equipa seu agente com um arsenal completo de equipamentos especializados — cada um afiado para uma tarefa específica na construção de software.

### Três categorias de ferramentas

---

As 31 ferramentas do OMP são organizadas em três grandes categorias, cada uma com um conjunto de responsabilidades que se complementa. Essa organização não é apenas conceitual — ela define como o agente acessa, usa e combina as ferramentas durante uma sessão.

### Ferramentas de Arquivo (7 ferramentas)

---

O casco de qualquer projeto de software são seus arquivos. O OMP oferece sete ferramentas dedicadas a manipular esse casco com precisão cirúrgica.

**read** — A ferramenta de leitura mais completa do mercado. Não apenas lê arquivos inteiros: suporta offsets (linha inicial) e limits (quantidade de linhas), permitindo que o agente leia apenas a seção que precisa. Quando lê um diretório, retorna a lista de entradas. Para arquivos binários e PDFs, renderiza como anexo.

É a lupa do estaleiro — quando você precisa inspecionar uma viga específica na quilha, não precisa levantar o navio inteiro.

**write** — Gravação direta de conteúdo. Sobrescreve arquivos existentes ou cria novos. O agente deve usar `read` antes de `write` em arquivos existentes — isso garante que ele conhece o estado atual antes de modificá-lo.

É a fundição do estaleiro: quando uma peça precisa ser refeita do zero.

**edit** — Edições por substituição de string exata. O agente especifica `old_string` (o texto atual) e `new_string` (o texto desejado), e a ferramenta localiza e substitui. Se houver múltiplas ocorrências, o agente precisa fornecer mais contexto.

É a soldadora de precisão — corta exatamente onde deve, sem afetar a estrutura ao redor.

**ast\_edit** — Edições por âncoras de hash em vez de conteúdo completo. Em vez de enviar todo o bloco de código como `old_string`, o agente referencia um hash que identifica o bloco. Isso reduz drasticamente os tokens consumidos — blocos de código estável não precisam ser reenviados a cada edição.

É a âncora do estaleiro: segura a peça no lugar sem precisar descrevê-la inteira a cada ajuste.

**grep** — Busca por conteúdo dentro de arquivos usando regex. Retorna caminhos, numeração de linha e trechos destacados. O agente pode filtrar por padrão de arquivo.

É o sonar do estaleiro — varre o casco inteiro procurando uma solda específica.

**glob** — Busca por padrão de nome de arquivo. Encontra todos os arquivos que correspondem a uma expressão `glob`. Retorna caminhos ordenados por data de modificação.

É o inventário do estaleiro — lista todas as peças disponíveis por tipo e tamanho.

**ast\_grep** — Busca semântica em AST (Abstract Syntax Tree). Diferente do `grep` que busca texto bruto, o `ast_grep` entende a estrutura do código. Você pode buscar por padrões como “toda função que retorna Promise” ou “toda classe que herda de Error”.

É o raio-X do estaleiro — vê a estrutura interna do metal, não apenas a superfície.

## Ferramentas de Runtime (2 ferramentas)

---

Enquanto as ferramentas de arquivo operam sobre o casco estático do projeto, as ferramentas de runtime são o motor — elas fazem o código realmente rodar.

**bash** — Execução de comandos shell. Suporta timeout configurável, captura completa de `stdout/stderr`, e modos interativos para comandos que precisam de input do usuário. É o motor principal do estaleiro — quando você precisa rodar um build, executar testes ou instalar dependências.

**eval** — Avaliação inline de código Python ou JavaScript. Diferente do bash que executa comandos externos, o eval executa código diretamente no contexto do agente. Ideal para transformações de dados, cálculos rápidos ou prototipação de lógica antes de gravá-la em um arquivo.

## Ferramentas Avançadas (5+ ferramentas)

---

Aqui está onde o OMP se diferencia de qualquer outro harness. Essas ferramentas transformam o agente de um assistente de código em um verdadeiro engenheiro autônomo.

**LSP** — Language Server Protocol integrado. Oferece 14 operações de inteligência de código: rename, diagnostics, code actions, completions, hover, e mais. Quando o agente renomeia uma função, ele não apenas faz busca-e-substitui — ele atualiza todas as referências, imports e tipos que dependem dela.

É o radar do estaleiro — detecta problemas antes que eles se tornem falhas estruturais.

**debug** — Debug Adapter Protocol (DAP). 28 operações de depuração: attach em processos, breakpoints condicionais, stepping (step over/into/out), inspection de variáveis, evaluation de expressões. O agente pode anexar a um processo Python, Go ou C++ e depurá-lo como faria um humano — só que mais rápido.

**task** — Sistema de fan-out para subagentes. Permite que o agente decomponha tarefas complexas em workers paralelos, cada um com seu escopo e schema de resultado validado. É a tripulação do estaleiro — quando o projeto é grande demais para um único engenheiro.

**browser** — Automação de browser headless via Puppeteer e CDP (Chrome DevTools Protocol). Navega em páginas, preenche formulários, extrai dados, tira screenshots. Modo stealth disponível via relay extension para evitar detecção.

É o mergulhador do estaleiro — entra na água para inspecionar partes que o olho humano não alcança.

**computer** — Controle nativo do desktop. Gerencia janelas, captura screenshots do screen inteiro, lê a AX tree (árvore de acessibilidade) para entender a UI, e envia input nativo (teclado e mouse).

É o guindaste do estaleiro — move, posiciona e opera peças que estão fora do escopo do casco.

## As 31 ferramentas em detalhe

---

A tabela completa lista todas as ferramentas built-in do OMP, organizadas por categoria.

| #  | Ferramenta              | Categoria    | Descrição                              | Caso de Uso                               |
|----|-------------------------|--------------|----------------------------------------|-------------------------------------------|
| 1  | <code>read</code>       | Arquivo      | Leitura de arquivos com offset/limit   | Inspecionar código em linhas específicas  |
| 2  | <code>write</code>      | Arquivo      | Gravação direta de conteúdo            | Criar ou sobrescrever arquivos            |
| 3  | <code>edit</code>       | Arquivo      | Edição por <code>str_replace</code>    | Corrigir bugs, refatorar trechos          |
| 4  | <code>ast_edit</code>   | Arquivo      | Edição por âncoras de hash             | Editar blocos grandes com menos tokens    |
| 5  | <code>grep</code>       | Arquivo      | Busca por conteúdo (regex)             | Encontrar chamadas de função, padrões     |
| 6  | <code>glob</code>       | Arquivo      | Busca por padrão de nome               | Listar arquivos por tipo ou local         |
| 7  | <code>ast_grep</code>   | Arquivo      | Busca semântica em AST                 | Encontrar padrões estruturais no código   |
| 8  | <code>bash</code>       | Runtime      | Execução de comandos shell             | Builds, testes, instalações               |
| 9  | <code>eval</code>       | Runtime      | Avaliação inline de Python/JS          | Transformação de dados, cálculos          |
| 10 | <code>LSP</code>        | Avançada     | 14 operações de inteligência de código | Rename seguro, diagnósticos, code actions |
| 11 | <code>debug</code>      | Avançada     | 28 operações DAP                       | Depuração de processos nativos            |
| 12 | <code>task</code>       | Avançada     | Fan-out de subagentes                  | Decomposição de tarefas complexas         |
| 13 | <code>browser</code>    | Avançada     | Automação de browser headless          | Scraping, testes UI, automação web        |
| 14 | <code>computer</code>   | Avançada     | Controle nativo do desktop             | Automação de desktop, screenshots         |
| 15 | <code>ask</code>        | Interação    | Picker de opções interativo            | Decisões que dependem do usuário          |
| 16 | <code>web_search</code> | Conhecimento | Busca na web                           | Pesquisa de documentação, APIs            |
| 17 | <code>web_fetch</code>  | Conhecimento | Fetch de conteúdo web                  | Leitura de páginas, docs online           |
| 18 | <code>memory</code>     | Conhecimento | Gerenciamento de memória               | Retenção de contexto entre sessões        |
| 19 | <code>retain</code>     | Conhecimento | Armazenamento de contexto              | Salvar descobertas importantes            |

## Os 16 esquemas internos de URI

---

Além das ferramentas, o OMP utiliza 16 esquemas de URI internos para referenciar recursos de forma padronizada. Esses esquemas permitem que o agente acesse diferentes tipos de recursos com uma sintaxe unificada.

**pr://** — Referência a Pull Requests no GitHub.

**issue://** — Referência a Issues no GitHub.

**agent://** — Referência a subagentes em execução.

**skill://** — Referência a skills instaladas.

**ssh://** — Conexões SSH para execução remota.

**file://** — Referência a arquivos locais.

**url://** — Referência a URLs externas.

**mcp://** — Referência a servidores MCP conectados.

E outros 8 esquemas para recursos internos do harness.

Esses esquemas são como os diferentes tipos de documentos de um estaleiro — cada um tem sua formatação, seu protocolo de acesso, e sua finalidade específica.

## Fluxo de uso combinado

---

O poder real das ferramentas do OMP está na combinação. Veja um fluxo típico de refatoração.

```
# 1. Descobrir onde a função é usada
grep "processarPedido" --include "*.ts"

# 2. Ver a estrutura do código
ast_grep "function processarPedido($$$) { $$$ }"

# 3. Ler o arquivo completo
read src/services/pedido.ts

# 4. Fazer a edição com ast_edit (menos tokens)
ast_edit --hash "a3f8c2" --new "function processarPedido(dados: PedidoDT0):
Promise<Resultado>"

# 5. Rodar testes para validar
bash "npm test -- --grep 'processarPedido'"
```



```
# 6. Verificar se o LSP aprovou
LSP diagnostics src/services/pedido.ts
```

Cada passo usa a ferramenta mais adequada para a tarefa. O agente não usa bash para tudo — ele escolhe a ferramenta certa para cada etapa, economizando tokens e aumentando a precisão.

### Economia de tokens na prática

Note como cada ferramenta foi escolhida para minimizar o consumo de tokens.

| Ação                  | Ferramenta Alternativa (menos eficiente) | Ferramenta OMP (eficiente) | Economia          |
|-----------------------|------------------------------------------|----------------------------|-------------------|
| Encontrar tipo        | bash + grep manual                       | grep built-in              | ~40% menos tokens |
| Ler trecho específico | read arquivo inteiro                     | read com offset/limit      | ~70% menos tokens |
| Editar bloco          | edit com old_string completo             | ast_edit com hash          | ~61% menos tokens |
| Validar               | bash + tsc manual                        | LSP diagnostics            | ~50% menos tokens |

O resultado é uma sessão que consome menos tokens, é mais precisa, e produz menos erros. É a diferença entre um estaleiro que usa ferramentas manuais e um que usa equipamentos hidráulicos de precisão.

## Próximos Passos

---

Neste capítulo, você abriu o arsenal completo do OMP. As 31 ferramentas built-in não são apenas uma lista de funcionalidades — são o equipamento que transforma uma IA de conversação em um engenheiro de software autônomo.

Cada ferramenta tem seu papel, cada uma é otimizada para sua tarefa, e juntas elas cobrem todo o ciclo de vida do desenvolvimento: de inspecionar código existente a depurar processos em execução.

No próximo capítulo, você mergulhará fundo no sistema de hashline edits — o mecanismo que reduz tokens de saída em até 61% e que é o coração da eficiência do OMP.

Acesse a referência completa de ferramentas: <https://omp.sh/docs/tools>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

---

## Edições Hashline: Precisão com Menos Tokens

---

### O problema que consome recursos

---

No estaleiro digital onde construímos harnesses de IA, existe um problema antigo que consome recursos como um motor sem eficiência: **redundantemente enviamos o código completo** toda vez que queremos fazer uma pequena alteração.

Imagine um mestre de estaleiro que, para trocar um parafuso no casco, precisasse reconstruir todo o navio. Isso seria absurdo — e é exatamente o que acontece quando usamos métodos tradicionais de edição.

O **hashline edit** é a âncora que estabiliza nossa navegação. Em vez de enviar o conteúdo completo de um arquivo, o sistema gera um **hash** (uma impressão digital criptográfica) de cada bloco de código.

Quando queremos modificar algo, bastamos referenciar o hash — como um GPS que diz exatamente onde estamos no oceano digital, sem precisar descrever toda a rota.

### O fluxo tradicional: tokens desperdiçados

---

Quando um agente de IA edita código, o fluxo tradicional é:

1. Ler o arquivo completo (N tokens)
2. Identificar o que mudar
3. Reescrever o arquivo inteiro (N tokens)

Isso significa que, para uma alteração de 10 linhas em um arquivo de 500 linhas, pagamos **1000 tokens** — o dobro do necessário. No estaleiro naval, seria como pintar todo o casco para trocar uma única tinta.

## A solução: hashes como âncoras

O hashline edit transforma esse fluxo.

1. Gerar hashes dos blocos existentes (custo: zero)
2. Referenciar o hash do bloco-alvo (custo: ~10 tokens)
3. Enviar apenas a alteração (custo: M tokens)

Agora, para aquela mesma alteração de 10 linhas, pagamos apenas **~10 tokens** — uma redução de **99%** no custo de edição.

## Como funciona o hash

O hash é gerado a partir do conteúdo do bloco. Se o bloco não mudou, o hash permanece idêntico. Isso cria um sistema de **referência estável**.

**Antes:** “Edite a função `processar_dados()` na linha 42”

**Depois:** “Edite o bloco com hash `a1b2c3d4`”

A vantagem é que o hash não depende de números de linha (que mudam com edições) nem do conteúdo completo (que consome tokens).

## Comparação com `str_replace`

O método `str_replace` (substituição de strings) é como tentar encontrar uma agulha no palheiro.

| Método                   | Tokens por Edição | Confiabilidade | Colisões                       |
|--------------------------|-------------------|----------------|--------------------------------|
| Conteúdo completo        | ~1000             | Alta           | Zero                           |
| <code>str_replace</code> | ~200              | Média          | Possíveis                      |
| Hashline                 | ~50               | Alta           | Criptograficamente improváveis |

O `str_replace` pode falhar quando o mesmo texto aparece múltiplas vezes, quando o texto contém caracteres especiais, ou quando o contexto é ambíguo.

O hash resolve isso porque cada bloco único gera um hash único — como um GPS que mostra coordenadas exatas, não descrições vagas.

## A economia por modelo

Os números são impressionantes.

| Modelo           | Tokens Antes | Tokens Depois | Redução |
|------------------|--------------|---------------|---------|
| Grok 4 Fast      | 10.000       | 3.900         | 61%     |
| Grok Code Fast 1 | 10.000       | 3.170         | 68.3%   |
| Gemini 3 Flash   | 10.000       | 5.200         | 48%     |

A métrica chave: o Grok Code Fast 1 apresentou um **10x lift** — a taxa de sucesso em edições subiu de 6.7% (com `str_replace`) para 68.3% (com `hashline`).

## Estrutura de um hash

O hash no Oh My Pi segue o padrão `##/<hash-8-chars>`.

Exemplo de blocos com hashes.

```
##/a1b2c3d4
def processar_dados(dados):
    resultado = []
    for item in dados:
        if item.get("valido"):
            resultado.append(transformar(item))
    return resultado
##/a1b2c3d4
```

## Sintaxe de edição

Para editar um bloco, o agente envia.

```
##/a1b2c3d4 (replace: <hash-do-bloco-novo>)
def processar_dados(dados):
    resultado = []
    for item in dados:
        if item.get("valido") and item.get("prioridade") > 5:
```

```
        resultado.append(transformar(item))
    return resultado
#//noval234
```

## Exemplo prático

**Cenário:** Queremos adicionar validação de tipos na função.

### Método Tradicional (str\_replace):

```
Substituir:
def processar_dados(dados):
    resultado = []
    for item in dados:
        if item.get("valido"):
            resultado.append(transformar(item))
    return resultado

Por:
def processar_dados(dados: list[dict]) -> list:
    resultado = []
    for item in dados:
        if item.get("valido"):
            resultado.append(transformar(item))
    return resultado
```

**Custo:** ~150 tokens

### Método Hashline:

```
#//alb2c3d4 (replace: <hash-do-bloco-novo>)
def processar_dados(dados: list[dict]) -> list:
    resultado = []
    for item in dados:
        if item.get("valido"):
            resultado.append(transformar(item))
    return resultado
#//fle2d3c4
```

**Custo:** ~80 tokens

## Implementação no Oh My Pi

```
import hashlib

def gerar_hash(conteudo: str) -> str:
    """Gera hash curto de 8 caracteres para um bloco de código."""
    return hashlib.sha256(conteudo.encode()).hexdigest()[:8]

def identificar_blocos(arquivo: str) -> dict:
    """Identifica blocos delimitados por hashes e retorna mapeamento."""
    blocos = {}
    linhas = arquivo.split("\n")
    hash_atual = None
    inicio = 0

    for i, linha in enumerate(linhas):
        if linha.startswith("#//") and len(linha) == 11:
            if hash_atual is None:
                hash_atual = linha[3:]
                inicio = i + 1
            else:
                blocos[hash_atual] = {
                    "inicio": inicio,
                    "fim": i,
                    "conteudo": "\n".join(linhas[inicio:i])
                }
                hash_atual = None

    return blocos
```

## Quando usar hashline edits

| Cenário                         | Recomendação              |
|---------------------------------|---------------------------|
| Edição de função existente      | ✓ Hashline                |
| Adição de nova função           | ✓ Hashline                |
| Modificação de múltiplos blocos | ✓ Hashline                |
| Reescrita completa do arquivo   | ✗ Conteúdo completo       |
| Busca e substituição simples    | ⚠ str_replace pode bastar |

## Caso de uso: refatoração de código

---

Imagine que você é o mestre de estaleiro e precisa modernizar o motor do navio.

### Antes (Código Legado):

```
#!/motor001
def calcular_velocidade(distancia, tempo):
    return distancia / tempo
#!/motor001
```

### Depois (Código Modernizado com Hashline):

```
#!/motor001 (replace: <novo-hash>)
def calcular_velocidade(distancia: float, tempo: float) -> float:
    """Calcula a velocidade média em km/h.

    Args:
        distancia: Distância percorrida em km
        tempo: Tempo gasto em horas

    Returns:
        Velocidade média em km/h

    Raises:
        ValueError: Se tempo for zero ou negativo
    """
    if tempo <= 0:
        raise ValueError("Tempo deve ser positivo")
    return distancia / tempo
#!/motor-novo
```

**Economia:** Em vez de enviar todo o arquivo (que pode ter centenas de linhas), enviamos apenas o hash do bloco (~8 tokens) + a alteração (~50 tokens) = ~58 tokens, em vez de ~500+ tokens.

## Dicas do Mestre de Estaleiro

---

**Seja preciso.** Um hash errado edita o bloco errado. Sempre verifique antes de enviar.

**Prefira blocos pequenos.** Blocos menores = hashes mais específicos = menos ambiguidade.

**Use para iterações rápidas.** Quando você está testando múltiplas versões, hashline reduz o custo drasticamente.



**Combine com outras ferramentas.** Use hashline para edições pontuais e outros métodos para reestruturações maiores.

## Próximos Passos

---

Assim como um mestre de estaleiro experiente sabe exatamente qual parte do casco precisa de reparo sem precisar desmontar todo o navio, o hashline edit permite que agentes de IA façam edições precisas com o mínimo de recursos.

Os números falam por si: **61% menos tokens** com o Grok 4 Fast, **10x de melhoria** na acurácia com o Grok Code Fast 1, e **5 pontos percentuais** de vantagem sobre str\_replace com o Gemini 3 Flash.

No próximo capítulo, vamos explorar como integrar essas edições em fluxos de trabalho mais complexos, combinando hashline com outras técnicas de otimização.

Lembre-se: no estaleiro digital, **precisão é mais valiosa que quantidade**. Cada token economizado é um nó a mais no cabo que segura o navio no porto.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

---

